Lab 2

3.2

Submit these background exercises with the lab report.

3.2.1

1: $y(t) = \frac{d}{dt}x(t)$

2: $y(t) = \int\limits_{-\infty}^{t} x(\tau)\,d\tau$

1

Formulate a discrete-time system that approximates the continuous-time function.

✓ Answer

With T as our sampling period, and $t-1$ representing the previous discrete sample,

1: $y'[t] = \frac{y[t]-y[t-1]}{T}$

2: $Y[t] = T\sum\limits_{-\infty}^{t} y[t]$

2

Write down the difference equation that describes your discrete-time system. Your difference equation should be in closed form, i.e. no summations.

✓ Answer

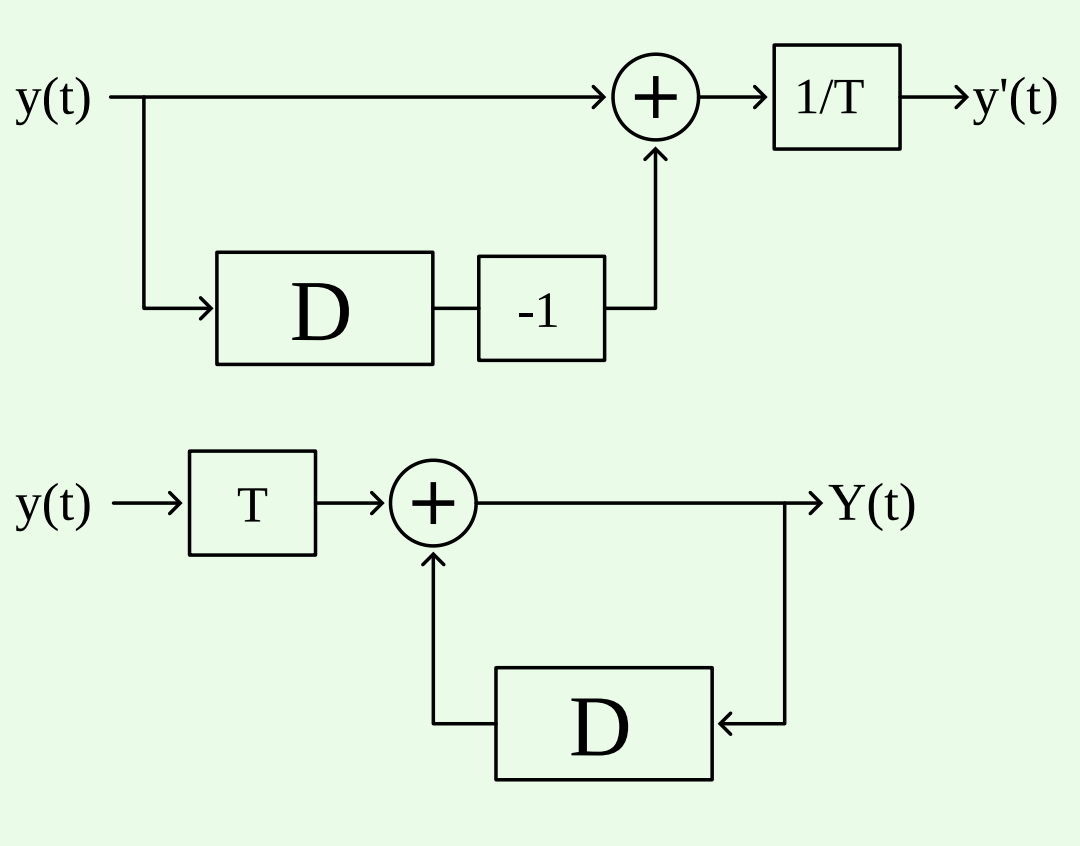
1: $y'[t] = \frac{y[t]-y[t-1]}{T}$

2: $Y[t] = Y[t-1] + Ty[t]$

3

Draw a block diagram of your discrete-time system as in Figure 3.1.

✓ Answer



3.2.2

1

write a difference equation

✓ Answer

$y[t] = \frac{1}{3}(x[t] + x[t-1] + x[t-2])$

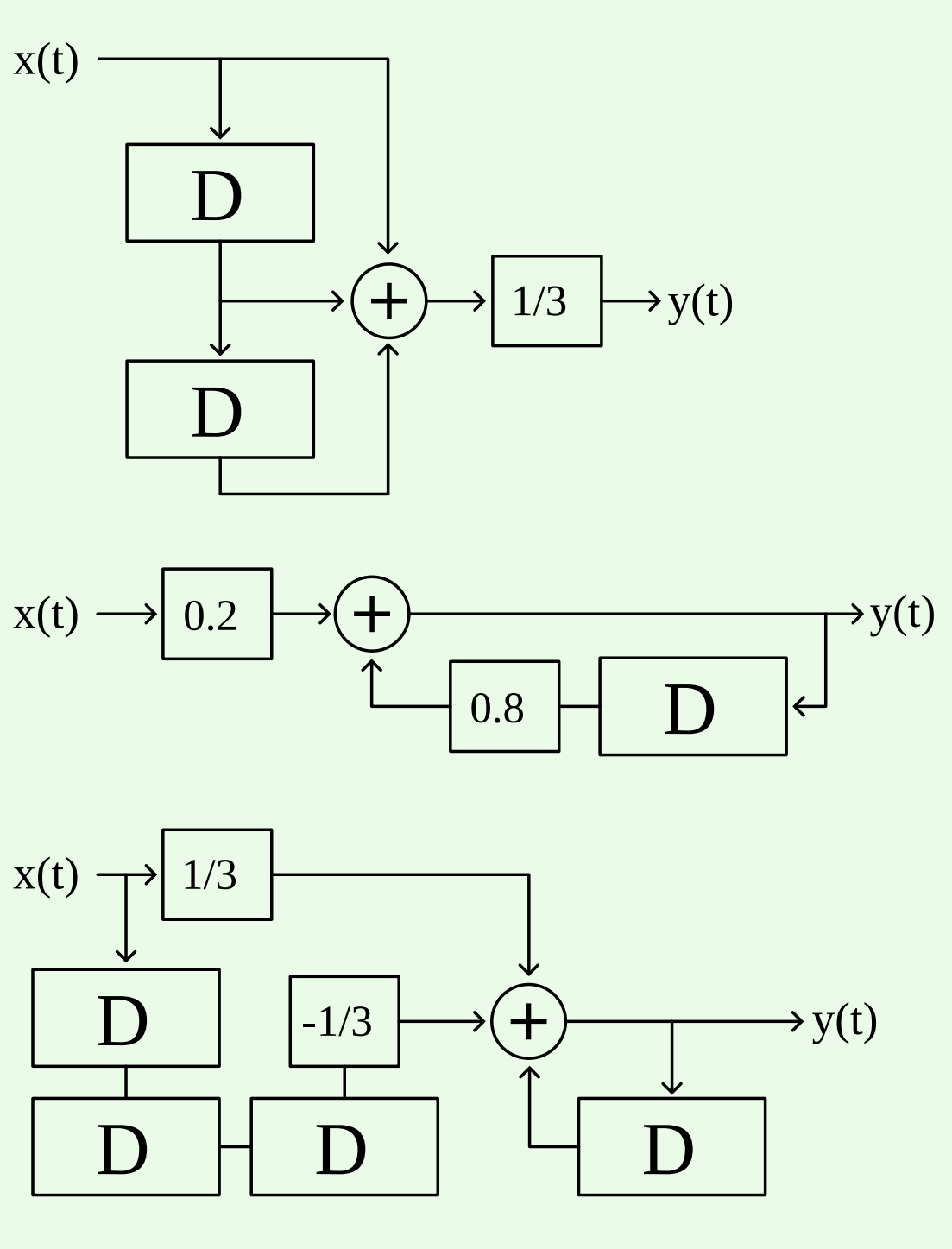
$y[t] = 0.8y[t-1] + 0.2x[t]$

$y[t] = y[t-1] + \frac{1}{3}(x[t] - x[t-3])$

2

draw a system diagram

✓ Answer



3

calculate the impulse response.

✓ Answer

Assuming $x[t] = \delta[t]$ and $x[t] = 0 : t < 0$, we can solve for $y[t]$.

$$y[t] = \begin{cases} \frac{1}{3} & t = 0, 1, 2 \\ 0 & \text{else} \end{cases}$$

$$y[t] = u(t)0.2(0.8)^t$$

$$y[t] = \begin{cases} \frac{1}{3} & t = 0, 1, 2 \\ 0 & \text{else} \end{cases}$$

4

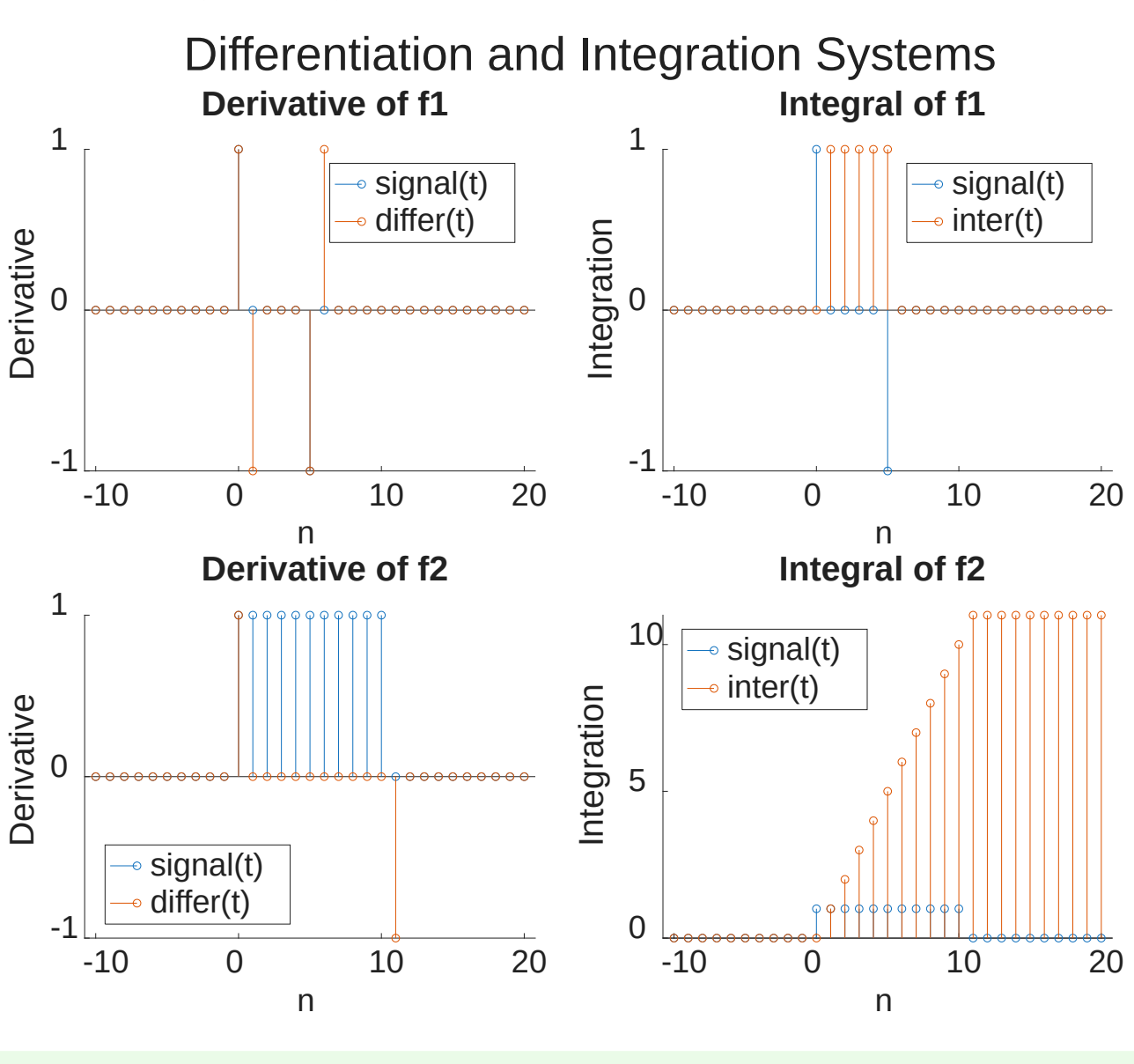
Explain why methods (3.3) and (3.5) are known as moving averages.

✓ Answer

Because it only takes into account the current and previous few values for the current average calculation, while equally weighting those few previous values. In our case, we are performing a moving average of the last 3 recordings.

3.3

Submit printouts of your Matlab code and hardcopies containing the input and output signals. Discuss the stability of these systems.



✓ Answer

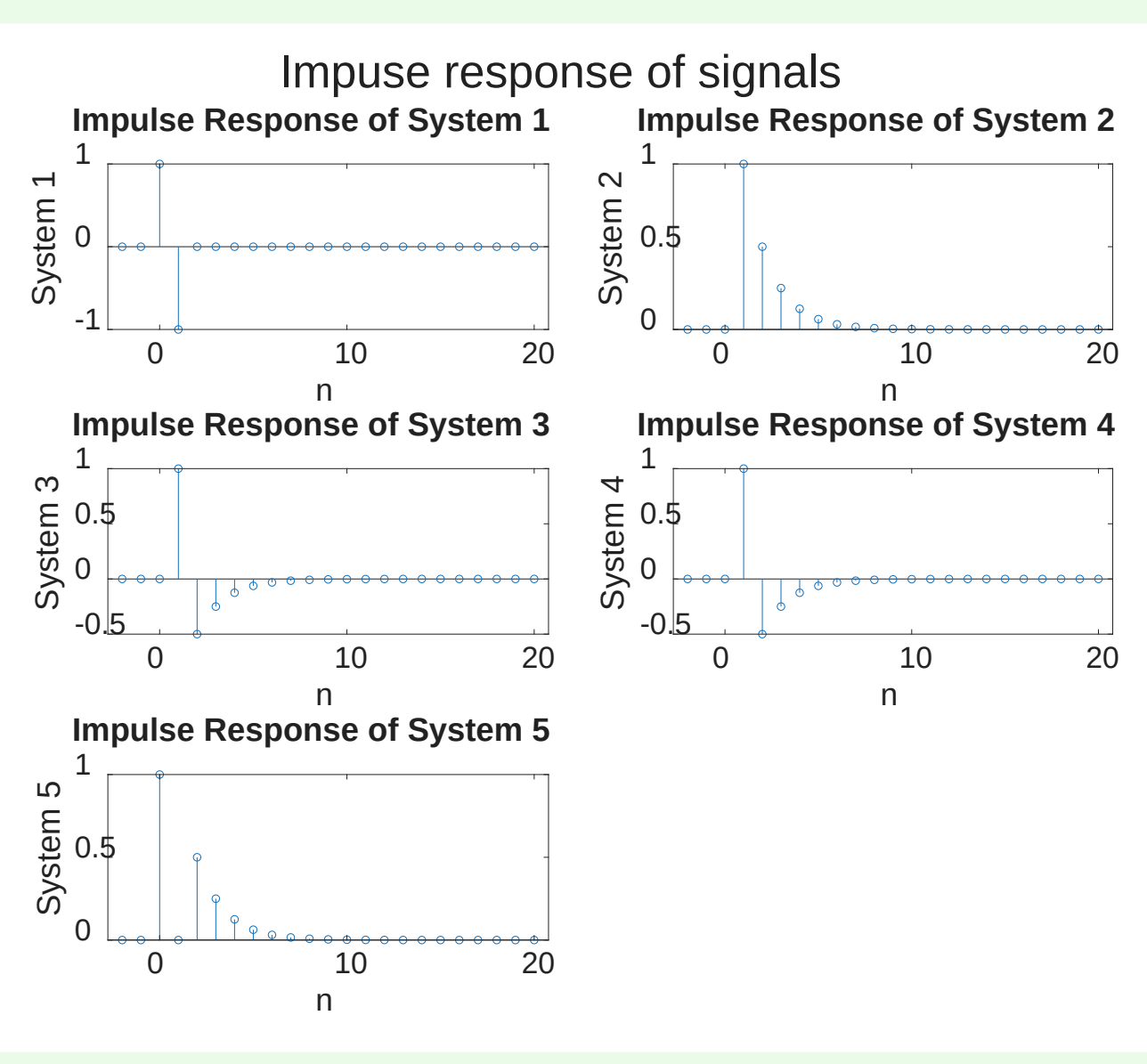
They all appear to be quite stable, however the integral of the second function does not collapse to 0 as time goes to infinity, meaning it is likely marginally stable.

3.4

For each of the five systems, draw and submit a system diagram (use only delays, multiplications, and additions as in Figure 3.1). Also submit plots of each impulse response. Discuss your observations.

✓ Answer

$x(n) = y(n) - \frac{1}{2}y(n-1)$



✓ Answer

Systems 3 and 4 appear the same since the signals are LTIC, meaning that the order of the composition of the systems do not matter, making both graphs appear the same.

System 5 appears as expected, as it is the sum of systems 1 and 2.

All systems appear to be stable.

3.5

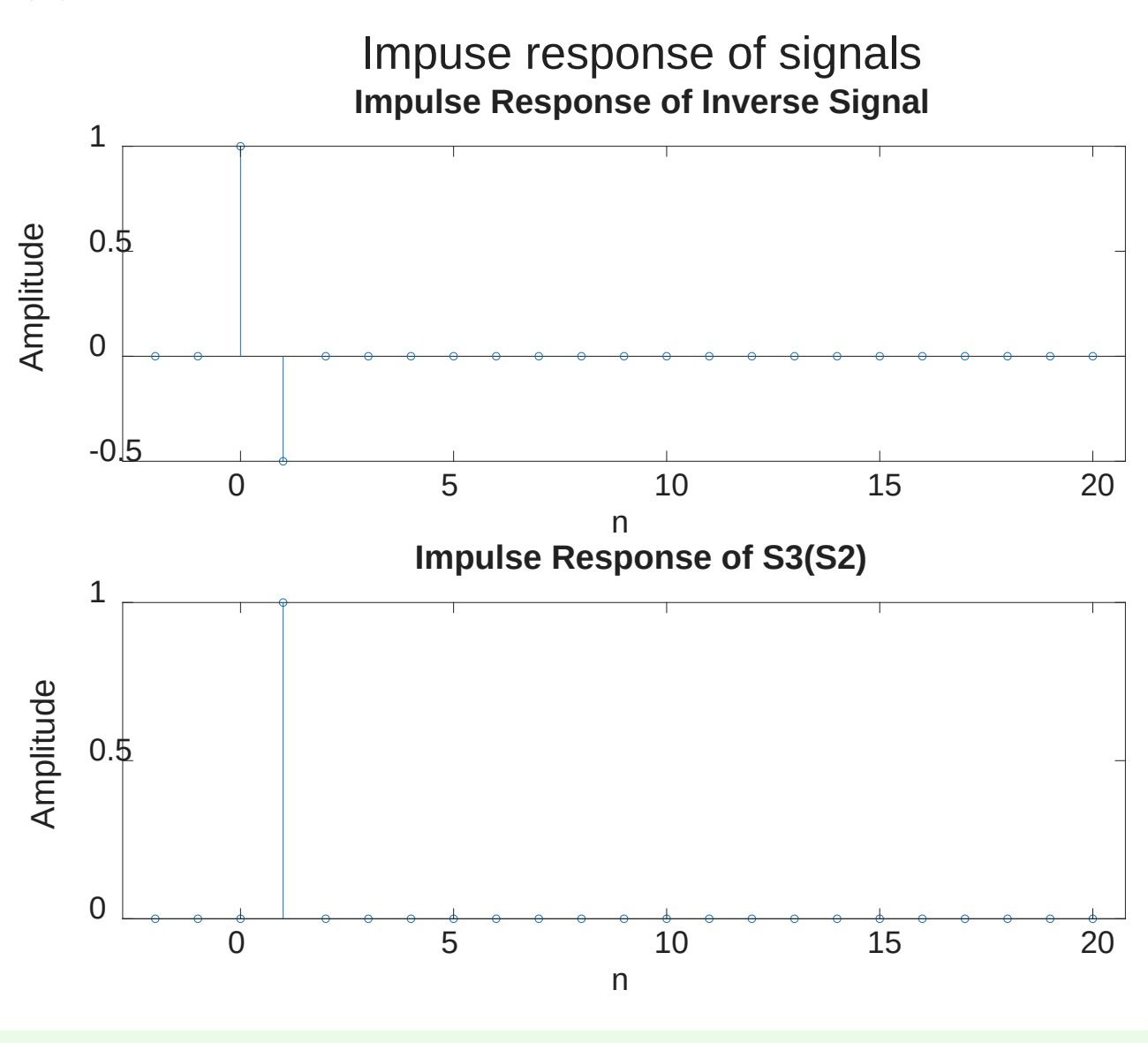
How do the filters change the sound of the audio signals? Explain your observations.

✓ Answer

The first filter make the audio sound more tinny and high pitched, similar to a low-cut filter. The second filter made the audio sound more bass-heavy and full.

3.6

Draw a system diagram for the system S_3, and submit plots of the impulse responses for S_3 and $S_3(S_2)$.

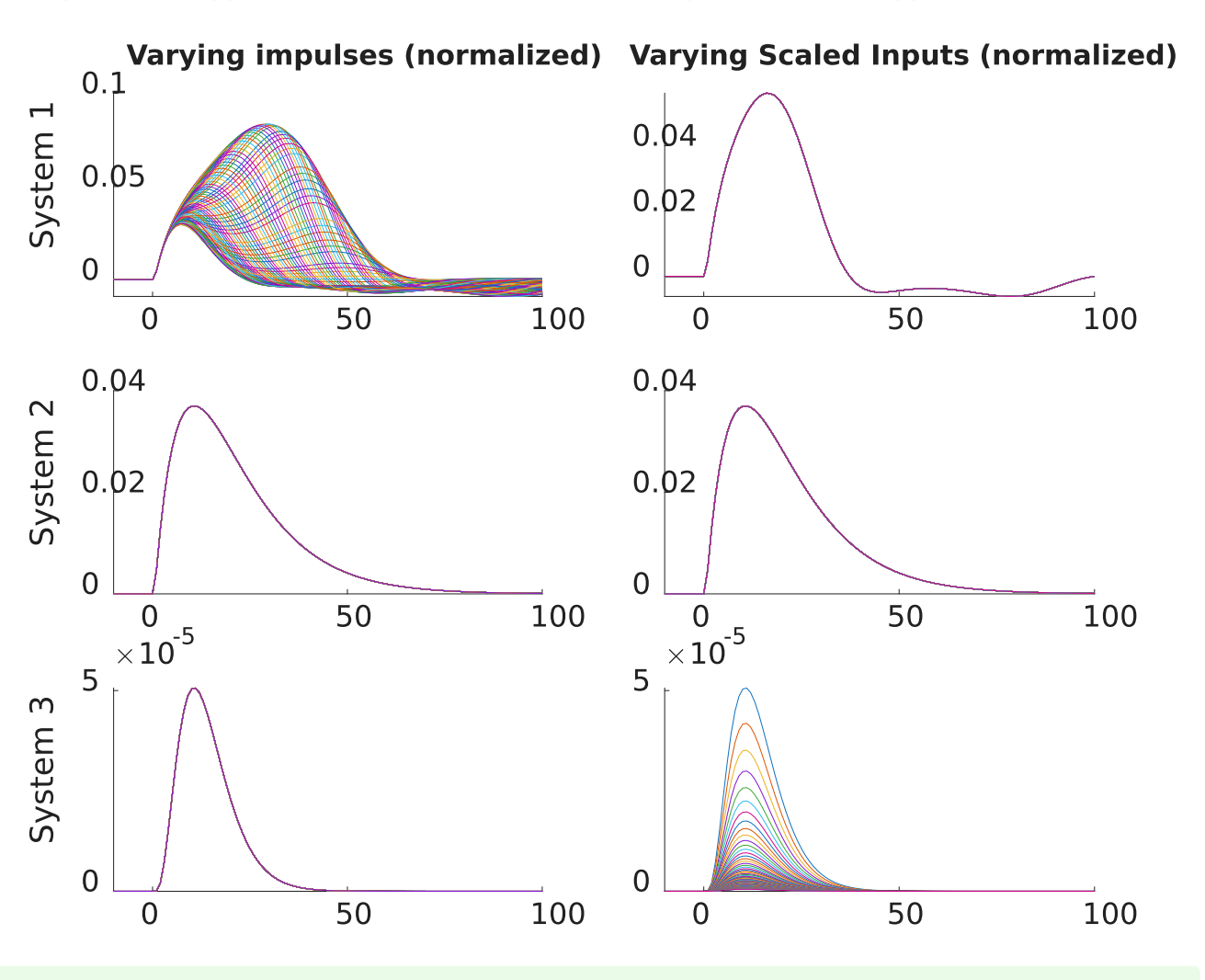


✓ Answer

The composition of the original function and its inverse appears to have a one sample delay since the inverse is LTIC, meaning it cannot look into the future for the next sample in order to reverse the previous LTIC system. The system we obtained is exactly as formatted within the lab, with our $b = 0.5$.

3.7

State which system is non-linear, and which system is time-varying. Submit plots of input/output signal pairs that support your conclusions. Indicate on the plots why they support your conclusions.



✓ Answer

In the first column, we varied where in the sample we initiate the impulse response, then normalized the starting position to be 0, as they should all be the same if it is time-invariant.

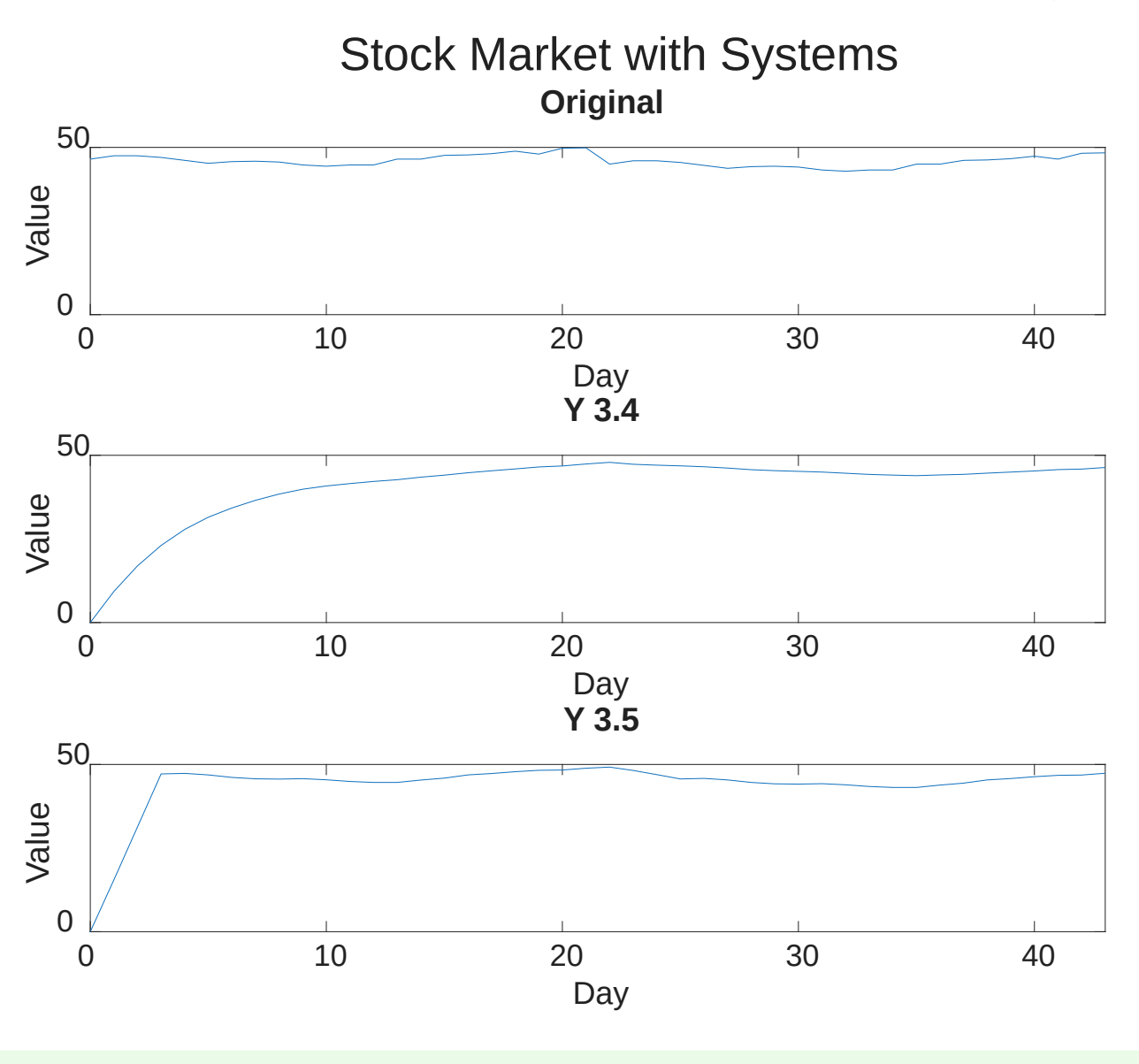
The first system clearly does not behave the same with a varying time input, meaning the system is time-variant. The other two systems behave identially regardless of the input time.

In the second column, we scaled the input impulse before passing the samples into the system, then unscaled the output back to what should be the same if the system were linear on the inputs.

The third system displays variance across scales in its output, meaning it is not linear. The other two systems behave as expected if linear, as the outputs all remain identical.

3.8

Submit your plots of the original and filtered exchange-rates. Discuss the advantages and disadvantages of the two filters. Can you suggest a better method for initializing the filter outputs?



✓ Answer

The first filter takes some time to "warm up" before it has a good reading of the rolling average whereas the second system has less of this issue. Instead of zeroing the inputs prior to our zero point, we can set them equal to the first data point, making the average start immediately at a reasonable value.

Code

`dirac.m`

```
% Define dirac function for use later
function result = dirac(x)
    result = x == 0;
end
```

`heviside.m`

```
% Define Heviside function for ease of use later
function result = heviside(x)
    result = x >= 0;
end
```

`helpers.m`

```
% A function to help set our defaults for all plots
function helpers = helpers(f)
    % Disables LaTeX rendering, as it is not supported by the export function
well
    set(f,'defaultTextInterpreter','none')
    set(f,'defaultAxesTickLabelInterpreter','none')
    set(f,'defaultLegendInterpreter','none')
    % Increase Default font size
    defaultFontSize = 25;
    set(f, 'defaultTextFontSize', defaultFontSize);
    set(f, 'defaultAxesFontSize', defaultFontSize);
    set(f, 'defaultLegendFontSize', defaultFontSize);
    % Increase Canvas size
    f.Position(3) = 1000;
    f.Position(4) = 800;
    % Disable slow output warning
    warning('off', 'MATLAB:print:ContentTypeImageSuggested')
    % Return our font size
    helpers = defaultFontSize;

end
```

`reduce.m`

```
% Define a simple reducer function for use in signals later as it is handy
% https://learn-functional-programming.com/chapters/4.7-sequence-
transformations/
function result = reduce(data, reduceF, init)
    result = zeros(1, length(data) + 1);
    result(1) = init;
    for i = 1:length(data)
        result(i+1) = reduceF(result(i), data(i));
    end
end
```

`q33.m`

```
% Define our differential and integrator functions
differ = @(s, T) (s(1:numel(s)) - [0, s(1:numel(s)-1)])/T;
inter = @(s, T) reduce(s, @(p, n) p + T*n, 0);

% Define our input functions
f1 = @(x) dirac(x) - dirac(x - 5);
f2 = @(x, N) heviside(x) - heviside(x - (N + 1));

% Define the domain
domain = -10:20;

% Sample our functions on our domain
f1x = f1(domain);
f2x = f2(domain, 10);

% Apply our systems to our input functions
df1x = differ(f1x, 1);
if1x = inter(f1x, 1);
df2x = differ(f2x, 1);
if2x = inter(f2x, 1);

% Set up canvas
f = figure;
ff = helpers(f);
t = tiledlayout(2,2);

% Plot Derivative of f1
nexttile
hold on
stem(domain, f1x)
stem(domain, df1x)
xlabel("n", FontSize=ff)
ylabel("Derivative")
title("Derivative of f1")
% Label all functions
legend( ...
    'signal(t)', ...
    'differ(t)', ...
    Location='best', ...
    FontSize=ff )

% Plot Integral of f1
nexttile
hold on
stem(domain, f1x)
stem(domain, if1x(1:numel(domain)))
xlabel("n", FontSize=ff)
ylabel("Integration")
title("Integral of f1")
% Label all functions
legend( ...
    'signal(t)', ...
    'inter(t)', ...
    Location='best', ...
    FontSize=ff )

% Plot Derivative of f2
nexttile
hold on
stem(domain, f2x)
stem(domain, df2x)
xlabel("n", FontSize=ff)
ylabel("Derivative")
title("Derivative of f2")
% Label all functions
legend( ...
    'signal(t)', ...
    'differ(t)', ...
    Location='best', ...
    FontSize=ff )

% Plot Integral of f2
nexttile
hold on
stem(domain, f2x)
stem(domain, if2x(1:numel(domain)))
xlabel("n", FontSize=ff)
ylabel("Integration")
title("Integral of f2")
% Label all functions
legend( ...
    'signal(t)', ...
    'inter(t)', ...
    Location='best', ...
    FontSize=ff )

% Set labels on toplevel canvas
sgtitle("Differentiation and Integration Systems", FontSize=ff*1.5)

% Export graphic
exportgraphics(f, 'plots/1_3.3.svg', ContentType='vector')
```

`q34.m`

```
% Define our systems as defined in the lab
y36 = @(s) s(1:numel(s)) - [0, s(1:numel(s)-1)];
y37 = @(s) reduce(s, @(p,n) 0.5.*p + n, 0);

% Define our domain and sample our function (delta)
domain = -2:20;
impr = dirac(domain);

% Define our systems and our signals going through these systems
systems = {y36, y37, @(s) y36(y37(s)), @(s) y37(y36(s)), @(s) [y36(s),
0]+y37(s)};
signals = cellfun(@(x) x(impr), systems, UniformOutput=false);

% Set up canvas
f = figure;
ff = helpers(f);
t = tiledlayout(3,2);

% Plot each signal
for i = 1:numel(signals)
    nexttile
    stem(domain, signals{i}(1:numel(domain)))
    xlabel("n", FontSize=ff)
    ylabel(sprintf('System %d', i))
    title(sprintf('Impulse Response of System %d', i))
end

% Set labels on toplevel canvas
sgtitle("Impuse response of signals", FontSize=ff*1.5)

% Export graphic
exportgraphics(f, 'plots/1_3.4.svg', ContentType='vector')
```

`q35.m`

```
% Define our systems
y36 = @(s) s - [0, s(1:numel(s)-1)];
y37 = @(s) reduce(s, @(p,n) 0.5.*p + n, 0);

% Read the audio
[ audio, Fs ] = audioread("music.au");

% Apply the systems
a1 = y36(audio');
a2 = y37(audio');

% Save the outputs to mp3 files
audiowrite("plots/music.mp3", audio, Fs);
audiowrite("plots/music1.mp3", a1, Fs);
audiowrite("plots/music2.mp3", a2, Fs);
```

`q36.m`

```
% Define our system and its inverse
y37 = @(s) reduce(s, @(p,n) 0.5.*p + n, 0);
y38 = @(s) s(1:numel(s)) - [0, s(1:numel(s)-1)]/2;

% Define our domain and sample our function
domain = -2:20;
impr = dirac(domain);

% Define our systems to plot
systems = {y38, @(s) y38(y37(s))};
signals = cellfun(@(x) x(impr), systems, UniformOutput=false);
labels = {"Inverse Signal", "S3(S2)"};

% Set up canvas
f = figure;
ff = helpers(f);
t = tiledlayout(2,1);

% Iterate over each signal and plot it
for i = 1:numel(signals)
    nexttile
    stem(domain, signals{i}(1:numel(domain)))
    xlabel("n", FontSize=ff)
    ylabel("Amplitude")
    title(sprintf('Impulse Response of %s', labels{i}))
end

% Set labels on toplevel canvas
sgtitle("Impuse response of signals", FontSize=ff*1.5)

% Export graphic
exportgraphics(f, 'plots/1_3.6.svg', ContentType='vector')
```

`q37.m`

```
% Define our constants, inputs and outputs
systems = {@system1, @system2, @system3};
xlims = [-10, 100];
domain = -1000:1000;
offsets = -200:5:200;
scales = 1:0.1:10;

inputs_offset = arrayfun(@(offset) dirac(domain+offset), offsets,
UniformOutput=false);
inputs_scales = arrayfun(@(scale) dirac(domain) ./ scale, scales,
UniformOutput=false);

outputs_offset = cellfun(@(system) cellfun(@(inputs_offset)
system(inputs_offset), inputs_offset, UniformOutput=false), systems,
UniformOutput=false);
outputs_scales = cellfun(@(system) cellfun(@(inputs_scale)
system(inputs_scale), inputs_scales, UniformOutput=false), systems,
UniformOutput=false);

% Set up canvas
f = figure;
ff = helpers(f);
t = tiledlayout(3, 2);

% For each system
for system = 1:numel(systems)
    nexttile(t);
    hold on
    % Title for top row
    if system == 1
        title('Varying impulses (normalized)')
    end
    % Plot dirac offsets
    for offset = 1:numel(offsets)
        plot(domain+offsets(offset), outputs_offset{system}{offset});
        % If left column, add row label
        if offset == 1
            ylabel(sprintf('System %d', system))
        end
        xlim(xlims)
    end
    nexttile(t);
    hold on
    % Title for top row
    if system == 1
        title('Varying Scaled Inputs (normalized)')
    end
    % Plot scale offsets
    for scale = 1:numel(scales)
        plot(domain, outputs_scales{system}{scale} .* scales(scale));
        xlim(xlims)
    end
end

% Export graphic
exportgraphics(f, 'plots/1_3.7.svg', ContentType='vector')
```

`q38.m`

```
% Define our systems
y34 = @(s) reduce(s, @(p,n) 0.8.*p + 0.2*n, 0);
diff = @(s, T) (s(1:numel(s)) - [zeros(1, 3), s(1:numel(s)-3)]);
y35 = @(s) reduce(diff(s), @(p,n)p + n/3, 0);

% Load stocks
load("stockrates.mat", "rate")

% Define domain, systems, systems applied to data, and labels
domain = 0:numel(rate)-1;

systems = {@(s) s, y34, y35};
signals = cellfun(@(x) x(rate), systems, UniformOutput=false);
labels = {"Original", "Y 3.4", "Y 3.5"};

% Set up canvas
f = figure;
ff = helpers(f);
t = tiledlayout(3,1);

% Iterate over each signal and plot it
for i = 1:numel(signals)
    nexttile
    plot(domain, signals{i}(1:numel(domain)))
    xlabel("Day", FontSize=ff)
    ylabel("Value")
    ylim([0, 50])
    xlim([0,numel(rate)-1])
    title(sprintf('%s', labels{i}))
end

% Set labels on toplevel canvas
sgtitle("Stock Market with Systems", FontSize=ff*1.5)

% Export graphic
exportgraphics(f, 'plots/1_3.8.svg', ContentType='vector')
```